

CI/CD

*Github Action -
Teste Automatizados*

PAULO R T CÂNDIDO

Instalação Vitest

Repositório (Branch b1)

<https://github.com/prtcandido/ghadockerexemplo>

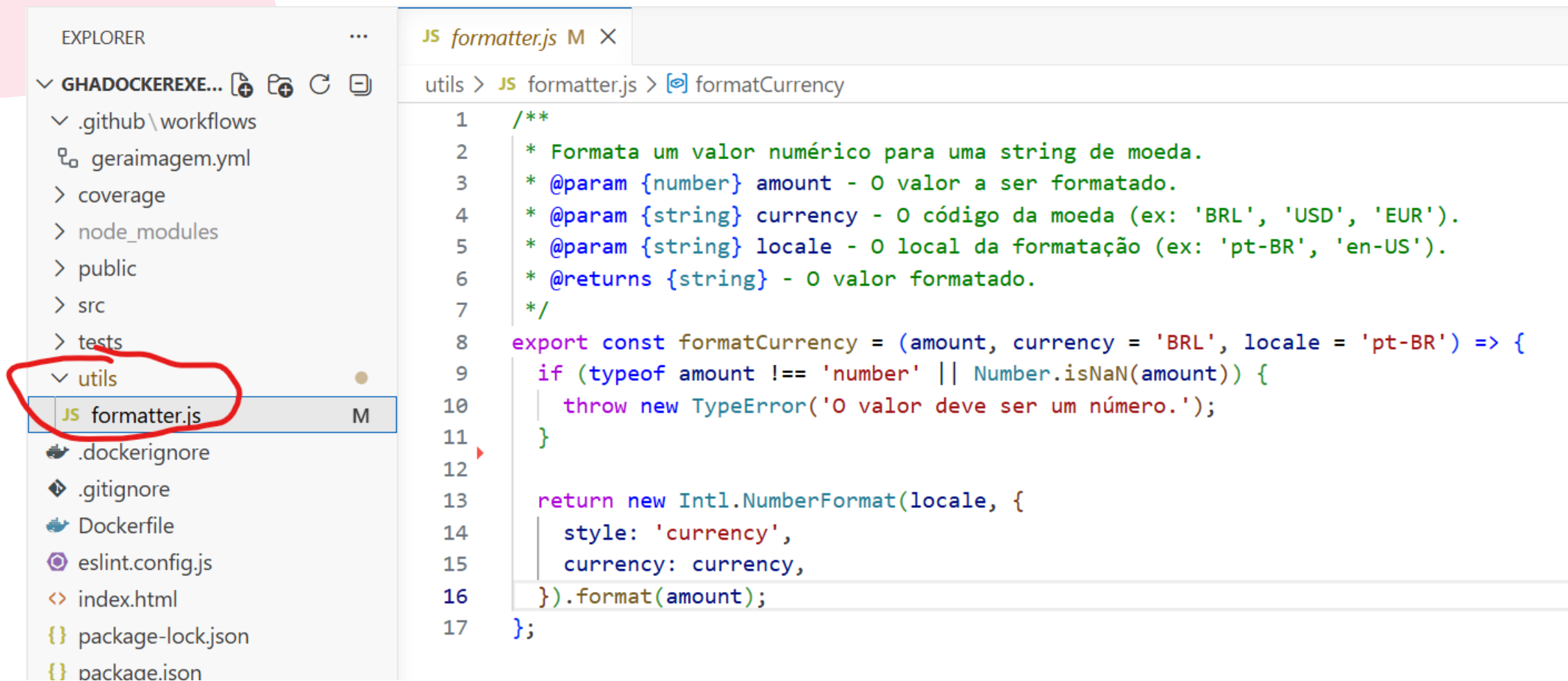
Instalação do Vitest

`npm install -D vitest`

Instalação do recurso para teste de cobertura (V8)

`npm install -D @vitest/coverage-v8`

Função a ser testada



EXPLORER

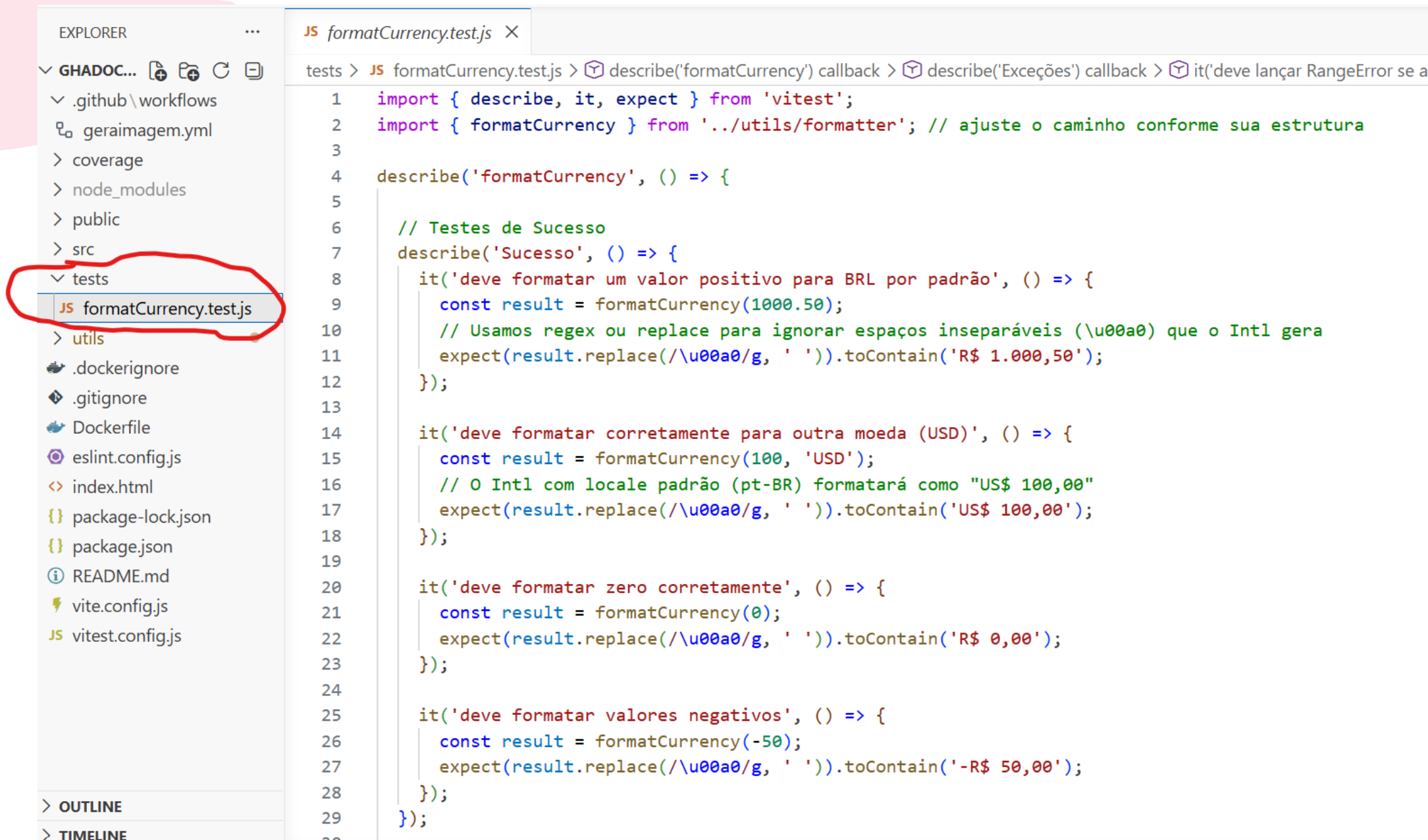
- GHADOCKEREXE...
- └ .github\workflows
- geraimagem.yml
- > coverage
- > node_modules
- > public
- > src
- > tests
- └ **utils**
 - JS formatter.js** M
- .dockerignore
- .gitignore
- Dockerfile
- eslint.config.js
- index.html
- package-lock.json
- package.json

JS formatter.js M X

utils > JS formatter.js > formatCurrency

```
1  /**
2   * Formata um valor numérico para uma string de moeda.
3   * @param {number} amount - O valor a ser formatado.
4   * @param {string} currency - O código da moeda (ex: 'BRL', 'USD', 'EUR').
5   * @param {string} locale - O local da formatação (ex: 'pt-BR', 'en-US').
6   * @returns {string} - O valor formatado.
7   */
8  export const formatCurrency = (amount, currency = 'BRL', locale = 'pt-BR') => {
9    if (typeof amount !== 'number' || Number.isNaN(amount)) {
10      throw new TypeError('O valor deve ser um número.');
```

Definição do teste



The image shows a VS Code editor interface. On the left, the Explorer sidebar displays a file tree with the following structure:

- GHADOC...
- .github\workflows
- geraimagem.yml
- coverage
- node_modules
- public
- src
- tests (highlighted with a red circle)
 - JS formatCurrency.test.js** (highlighted with a red circle)
 - utils
- .dockerignore
- .gitignore
- Dockerfile
- eslint.config.js
- index.html
- package-lock.json
- package.json
- README.md
- vite.config.js
- vitest.config.js

On the right, the editor displays the content of `JS formatCurrency.test.js`. The breadcrumb navigation at the top reads: `tests > JS formatCurrency.test.js > describe('formatCurrency') callback > describe('Exceções') callback > it('deve lançar RangeError se a`.

```
1 import { describe, it, expect } from 'vitest';
2 import { formatCurrency } from '../utils/formatter'; // ajuste o caminho conforme sua estrutura
3
4 describe('formatCurrency', () => {
5
6   // Testes de Sucesso
7   describe('Sucesso', () => {
8     it('deve formatar um valor positivo para BRL por padrão', () => {
9       const result = formatCurrency(1000.50);
10      // Usamos regex ou replace para ignorar espaços inseparáveis (\u00a0) que o Intl gera
11      expect(result.replace(/\u00a0/g, ' ')).toContain('R$ 1.000,50');
12    });
13
14    it('deve formatar corretamente para outra moeda (USD)', () => {
15      const result = formatCurrency(100, 'USD');
16      // O Intl com locale padrão (pt-BR) formatará como "US$ 100,00"
17      expect(result.replace(/\u00a0/g, ' ')).toContain('US$ 100,00');
18    });
19
20    it('deve formatar zero corretamente', () => {
21      const result = formatCurrency(0);
22      expect(result.replace(/\u00a0/g, ' ')).toContain('R$ 0,00');
23    });
24
25    it('deve formatar valores negativos', () => {
26      const result = formatCurrency(-50);
27      expect(result.replace(/\u00a0/g, ' ')).toContain('-R$ 50,00');
28    });
29  });
30 }
```

Definição do teste

> coverage

> node_modules

> public

> src

▼ tests

JS formatCurrency.test.js

> utils

🐳 .dockerignore

🔒 .gitignore

🐳 Dockerfile

⚙️ eslint.config.js

📄 index.html

📄 package-lock.json

📄 package.json

📖 README.md

⚡ vite.config.js

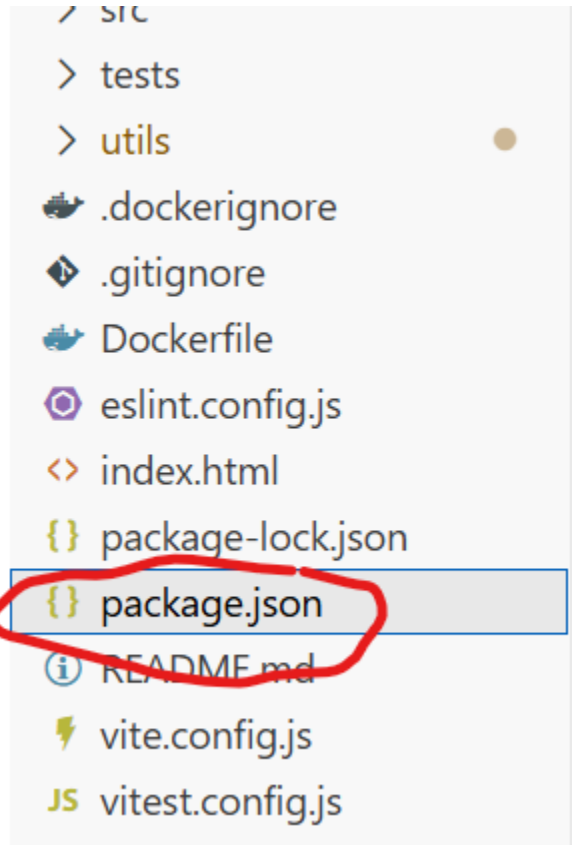
JS vitest.config.js

> OUTLINE

```
27 |     expect(result.replace(/\u00a0/g, ' ')).toContain('-R$ 50,00');
28 |   });
29 | });
30 |
31 | // Testes de Exceção
32 | describe('Exceções', () => {
33 |   it('deve lançar TypeError se o valor for uma string', () => {
34 |     expect(() => formatCurrency('100')).toThrow(TypeError);
35 |     expect(() => formatCurrency('100')).toThrow('O valor deve ser um número.');
```

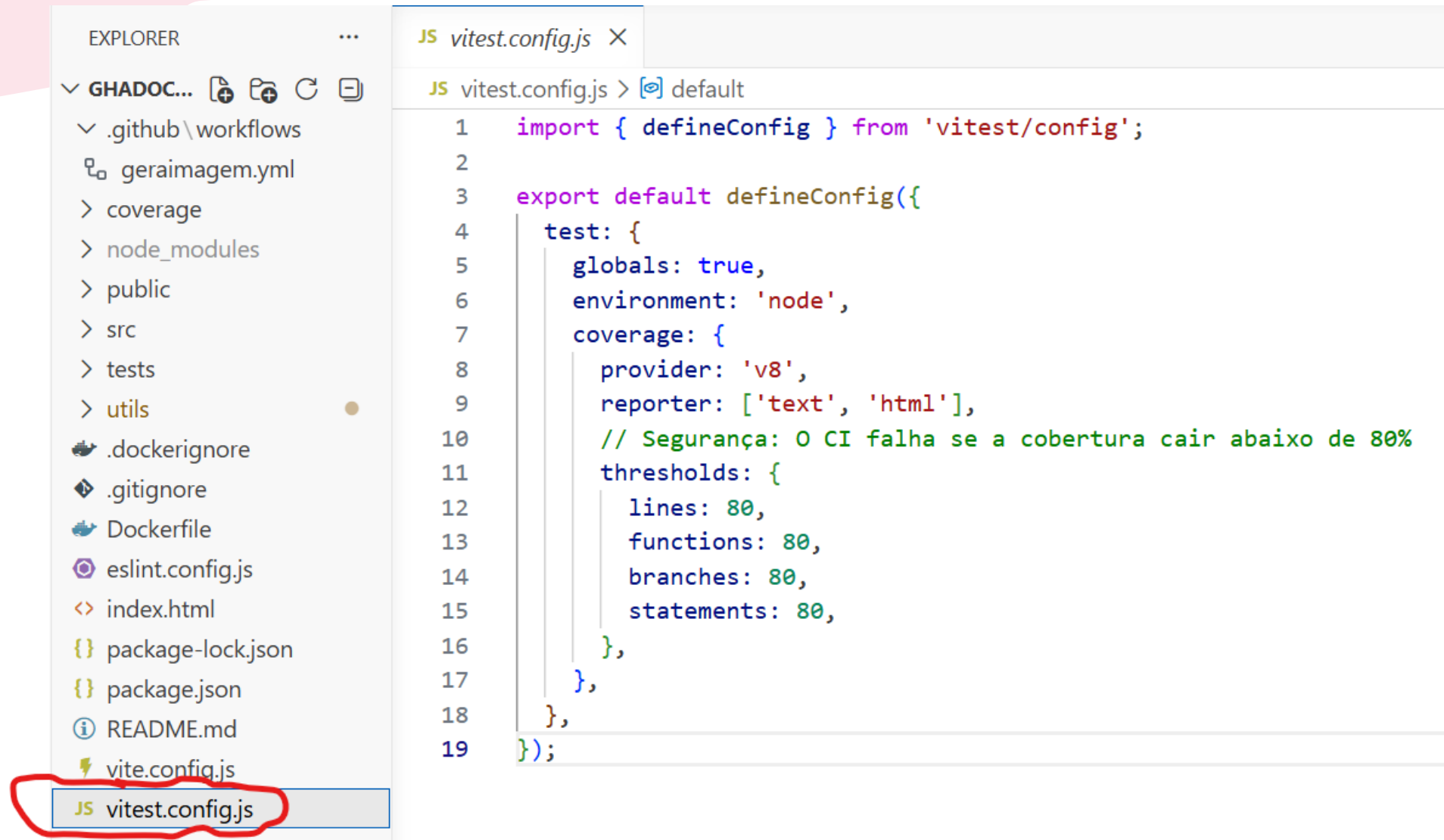
```
36 |   });
37 |
38 |   it('deve lançar TypeError se o valor for NaN', () => {
39 |     expect(() => formatCurrency(NaN)).toThrow(TypeError);
40 |   });
41 |
42 |   it('deve lançar TypeError se o valor for undefined ou null', () => {
43 |     expect(() => formatCurrency(undefined)).toThrow(TypeError);
44 |     expect(() => formatCurrency(null)).toThrow(TypeError);
45 |   });
46 |
47 |   it('deve lançar RangeError se a moeda for inválida', () => {
48 |     // O Intl lança RangeError internamente para códigos de moeda mal formatados
49 |     expect(() => formatCurrency(100, '123')).toThrow(RangeError);
50 |   });
51 | });
52 | });
```

Adicionar scripts



```
6  scripts : {
7    "dev": "vite",
8    "build": "vite build",
9    "lint": "eslint .",
10   "preview": "vite preview",
11   "test": "vitest",
12   "test:coverage": "vitest run --coverage"
13 },
14 "dependencies": {
15   "react": "^19.2.5",
16   "react-dom": "^19.2.5"
17 },
18 "devDependencies": {
19   "@eslint/js": "^10.0.1",
20   "@types/react": "^19.2.14",
```

Arquivo de configuração para os testes



The image shows a screenshot of a code editor (VS Code) with the Explorer panel on the left and the Editor panel on the right. The Explorer panel shows a file tree with various files and folders. The file `vitest.config.js` is highlighted with a red circle. The Editor panel shows the content of `vitest.config.js`, which is a JavaScript file defining the Vitest configuration.

EXPLORER

- GHADOC...
- .github\workflows
- geraimagem.yml
- coverage
- node_modules
- public
- src
- tests
- utils
- .dockerignore
- .gitignore
- Dockerfile
- eslint.config.js
- index.html
- package-lock.json
- package.json
- README.md
- vite.config.js
- JS vitest.config.js**

JS vitest.config.js

```
1 import { defineConfig } from 'vitest/config';
2
3 export default defineConfig({
4   test: {
5     globals: true,
6     environment: 'node',
7     coverage: {
8       provider: 'v8',
9       reporter: ['text', 'html'],
10      // Segurança: O CI falha se a cobertura cair abaixo de 80%
11      thresholds: {
12        lines: 80,
13        functions: 80,
14        branches: 80,
15        statements: 80,
16      },
17    },
18  },
19 });
```

Executar testes

Executa teste e ficar ativo aguardando alterações no código

`npm run test`

Executa teste e encerra

`npx vitest run`

Executa também o teste de cobertura

`npx vitest run --coverage`

Executa teste ESLint

`npm run lint`

Variáveis não definidas

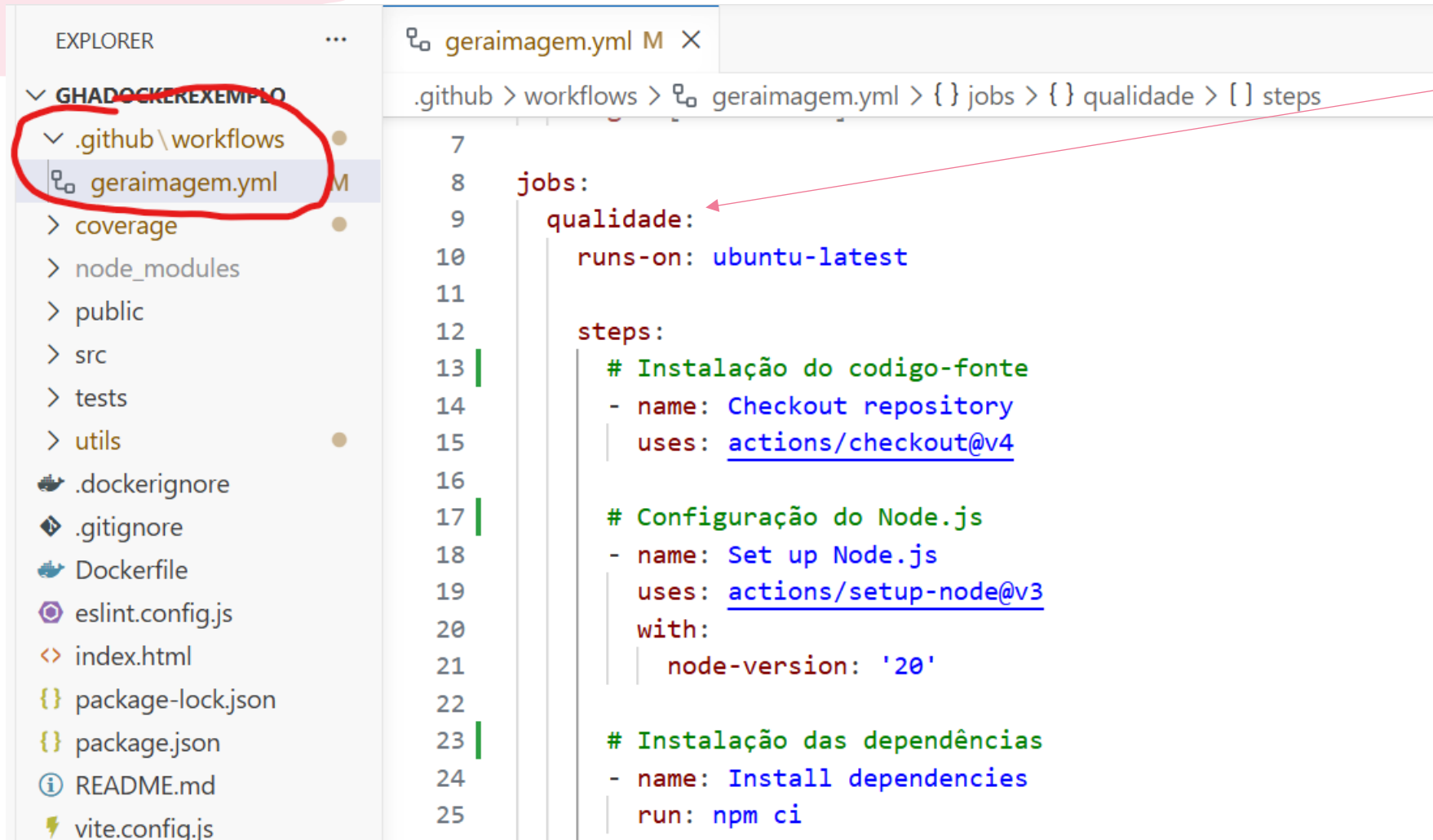
Variáveis não utilizadas

Loops infinitos

Código inalcançável

Comparações perigosas (`==` em vez de `===`)

Mudanças no Workflow



The screenshot shows the VS Code interface with the Explorer on the left and the Editor on the right. In the Explorer, the file `geraimagem.yml` is selected under the `.github/workflows` directory, which is circled in red. The Editor displays the content of `geraimagem.yml`, which defines a new job named `qualidade` for running tests. The breadcrumb navigation at the top of the editor reads: `.github > workflows > geraimagem.yml > {} jobs > {} qualidade > [] steps`. A red arrow points from the text on the right to the `qualidade:` key in the workflow file.

```
7
8 jobs:
9   qualidade:
10     runs-on: ubuntu-latest
11
12   steps:
13     # Instalação do código-fonte
14     - name: Checkout repository
15       uses: actions/checkout@v4
16
17     # Configuração do Node.js
18     - name: Set up Node.js
19       uses: actions/setup-node@v3
20       with:
21         node-version: '20'
22
23     # Instalação das dependências
24     - name: Install dependencies
25       run: npm ci
```

Novo JOB para
avaliar a qualidade
por meio de testes
automatizados

Mudanças no Workflow

EXPLORER

- GHADOC...
- .github\workflows
 - geraimagem.yml M
 - coverage
 - node_modules
 - public
 - src
 - tests
 - utils
- .dockerignore
- .gitignore
- Dockerfile
- eslint.config.js
- index.html
- package-lock.json
- package.json
- README.md
- vite.config.js
- vitest.config.js

geraimagem.yml M X

.github > workflows > geraimagem.yml > {} jobs > {} qualidade > [] steps

```
22
23 # Instalação das dependências
24 - name: Install dependencies
25   run: npm ci
26
27 # AUDITORIA DE SEGURANÇA
28 - name: Security Audit
29   run: npm audit --audit-level=high # Falha se houver vulnerabilidades graves
30
31 # ANÁLISE ESTÁTICA (LINT)
32 - name: Run Lint
33   run: npm run lint
34
35 # TESTES DE FUNÇÃO E COBERTURA
36 - name: Run Functional Tests
37   run: npm run test:coverage
38
39 build-and-push:
40   runs-on: ubuntu-latest
41   needs: qualidade
42
43 steps:
44   - name: Checkout repository
```

JOB que gera e publica a imagem Docker só é executado se o JOB de qualidade encerrar sem erros.

Exemplo de falha na qualidade

Summary

All jobs

qualidade

build-and-push

Run details

Usage

Workflow file

Triggered via push 41 minutes ago

Status

Total duration

Artifacts

prtcardido pushed 2f7629b b1

Failure

33s

—

geraimagem.yml

on: push

qualidade

20s

build-and-push

Annotations

2 errors and 1 warning

qualidade

Process completed with exit code 1.

qualidade: utils/formatter.js#L13

'x' is assigned a value but never used

qualidade

Node.js 20 actions are deprecated. The following actions are running on Node.js 20 and may not work as expected: actions/checkout@v4, actions/setup-node@v3. Act...

[Show more](#)